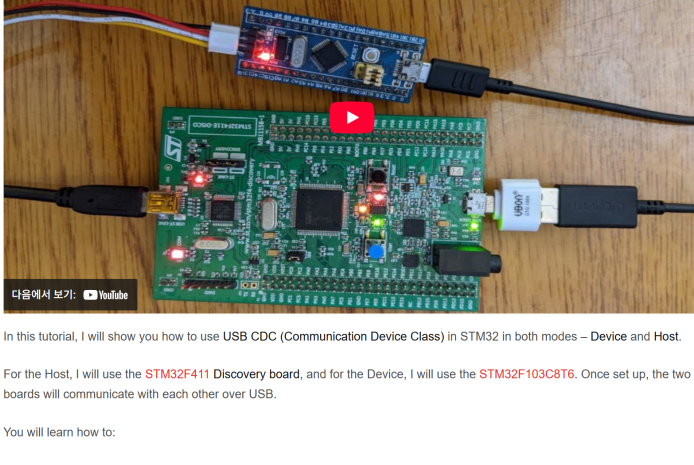


STM32 USB CDC Tutorial: Device and Host Mode Communication Example

Learn how to set up **STM32 USB CDC (Communication Device Class)** in both Device and Host modes. This tutorial covers a real-world **USB CDC example in STM32**, where an STM32F411 acts as a host and STM32F103 as a USB device. You'll learn about **CDC data transfer**, **VBUS control**, **pin configuration**, and **transmitting data** using the **USB OTG FS interface** and **STM32CubeMX + HAL**.

To better understand the steps, I've also created a detailed walkthrough video. You can follow along with the explanations below while watching the video here:



In this tutorial, I will show you how to use **USB CDC (Communication Device Class)** in STM32 in both modes – **Device** and **Host**. For the **Host**, I will use the **STM32F411 Discovery board**, and for the **Device**, I will use the **STM32F103C8T6**. Once set up, the two boards will communicate with each other over **USB**.

You will learn how to:

- Configure STM32 as a **USB CDC Device** (virtual COM port).
- Configure STM32 as a **USB CDC Host**.
- Exchange data between the two boards over **USB**.

By the end, you'll see a working example of **USB communication** between **STM32F411** and **STM32F103**.

Table Of Contents
1. What is USB CDC in STM32? (Overview of Communication Class)
2. STM32 USB CDC Host Setup (F411 Example)
3. STM32 USB CDC Device Configuration (F103 Setup)
4. Wiring Diagram
5. USB CDC Host Code (STM32 HAL Example)
6. USB CDC Device Code (CDC_Receive_FS Example)
7. RESULT
8. Other STM32 USB Tutorials
9. PROJECT DOWNLOAD
10. Project FAQs

What is USB CDC in STM32? (Overview of Communication Class)

USB CDC (Communication Device Class) is a standard defined by the **USB specification** to emulate **serial communication** ports over **USB**. In simple terms, it allows a **USB connection** to behave like a traditional **UART/RS232** serial port. This makes it very convenient for developers who want to exchange data without adding extra hardware like **USB-to-UART** converters.

In the context of **STM32 microcontrollers**, **USB CDC** is widely used to create **virtual COM ports**. With this feature, your **STM32 board** can communicate directly with a **PC** or even another **STM32 board** using just a **USB cable**. Depending on the project requirements, **STM32** can work in both **CDC Device mode** and **CDC Host mode**.

Why USB CDC is useful in STM32 projects:

- **Serial communication over USB** → No need for additional **UART** hardware.
- **Debugging and logging** → Developers often use **CDC** to send debug messages or logs to a **PC terminal**.
- **Sensor data streaming** → Useful for **real-time data transfer** from **STM32** to **PC** or another host.
- **Firmware flexibility** → Virtual **COM ports** are recognized by most operating systems without special drivers.
- **Two-way communication** → Enables both **sending** and **receiving data**, just like a standard **UART** port.

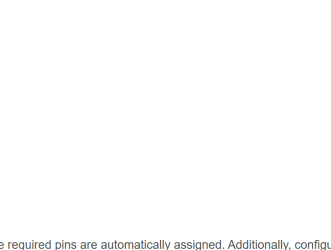
How STM32 simplifies USB CDC setup:

- **STM32CubeMX** provides **middleware support** for **USB CDC**, making the setup straightforward.
- **HAL libraries** handle **low-level USB protocol details**, so developers can focus on **application code**.
- **Autogenerated USB stack** includes all the necessary **descriptors** and **drivers** for **CDC operation**.
- **Cross-compatibility** → Works across many **STM32 families** (F1, F4, H7, etc.) with minimal changes.

STM32 USB CDC Host Setup (F411 Example)

In this part, we will configure the **STM32F411 Discovery board** to act as a **USB CDC Host**. The **Host** will detect the connected **STM32F103 Device** and establish communication over **USB**.

First, select the **USB_OTG_FS** in **Host Only** mode. Also, enable the **VBUS**, since the **Host** is responsible for supplying power to the connected **USB Device**.



Next, enable **USB_HOST** and set the class to **Communication Host Class**. Keep the remaining settings as default.

STM32 USB CDC Device Configuration (F103 Setup)

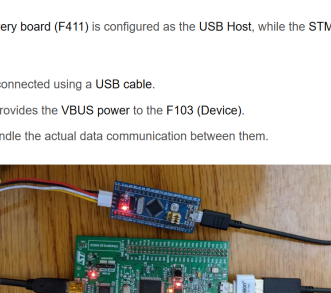
In this section, we will configure the **STM32F103** to work as a **USB CDC Device**. This setup will allow the **F103** to communicate with the **Host (STM32F411)** over **USB**, **sending** and **receiving data** as a **virtual COM port**.

Wiring Diagram

Wiring Diagram

In this setup, the **STM32F4 Discovery board (F411)** is configured as the **USB Host**, while the **STM32F103 (Blue Pill)** acts as the **USB Device**.

- The two boards are directly connected using a **USB cable**.
- The **F411 Discovery (Host)** provides the **VBUS** power to the **F103 (Device)**.
- The **USB D+ and D-** lines handle the actual data communication between them.



USB CDC Host Code (STM32 HAL Example)

Below are some **definitions**, that will be used in the code

```
1 extern USB_HandTypeDef hUSBHostFS;
2 extern ApplicationTypeDef Appl1_state;
3 extern USB_StatusTypeDef USBResult;
4
5 #define RX_BUFF_SIZE 64 /* Max Received data 3KB */
6
7 uint8_t CDC_RX_Buffer[RX_BUFF_SIZE];
8 uint8_t CDC_TX_Buffer[RX_BUFF_SIZE];
9
10 typedef enum {
```

The **CDC_HANDLE()** function will handle the data transmission in the host.

```
1 uint8_t i=0;
2 void CDC_HANDLE (void)
3 {
4     switch (CDC_STATE)
5     {
6     case CDC_STATE_IDLE:
7     {
8         USB_CDC_Stop(&hUSBHostFS);
9         Set len = sprintf ((char *)CDC_TX_Buffer, "DATA = %d", i);
10         if (USB_CDC_Transmit (&hUSBHostFS, CDC_TX_Buffer, len) == USB_OK)
```

- When the **Host** is **IDLE**, it will send the data to the **Device**.
- Once the data transfer is successful, it will change the **CDC state** to **RECEIVE** mode.
- In the **Receiving state**, Data will be received from the **device**, and stored in the **CDC_RX_Buffer**.
- The state will be again changed to **IDLE**, so that the next data can be transmitted.

This will create a **continuous communication loop** between the **Host (F411)** and the **Device (F103)**.

Inside the while loop, we will keep the **USB Host** running. When the connected device is ready, it starts handling **CDC communication**.

```
1 while (1)
2 {
3     /* USER CODE END WHILE */
4     MX_USB_HOST_Process();
5     if (Appl1_state == APPLICATION_READY)
6     {
7         CDC_HANDLE();
8     }
9 }
10 }
```

USB CDC Device Code (CDC_Receive_FS Example)

In the **CDC Device (STM32F103)**, the first step is to configure the **Line Coding**.

To do this, open the file: **USB_DEVICE => App => usbcd_cdc_if.c**

For handling data transfer, the important function is **CDC_Receive_FS**, which is responsible for receiving data from the **Host** and then sending it back.

```
1 static uint8_t CDC_Receive_FS(uint8_t* Buf, uint32_t *Len)
2 {
3     /* USER CODE BEGIN */
4     USB_CDC_SetReceiveBuffer(&hUSBDeviceFS, Buf[Buf]);
5     USB_CDC_ReceivePacket(&hUSBDeviceFS);
6     uint32_t len = *Len;
7     CDC_Transmit_FS (Buf, len);
8     return (USB_OK);
9     /* USER CODE END */
10 }
```

Here's what happens step by step:

1. The **USB stack** passes the received data into **CDC_Receive_FS**.
2. The function sets the receive buffer using **USB_CDC_SetReceiveBuffer()**.
3. The packet reception is confirmed with **USB_CDC_ReceivePacket()**.
4. The length of the received data is stored in **len**.
5. Finally, the same data is sent back to the **Host** using **CDC_Transmit_FS()**.

Basically, whenever the device gets the receive request, it will receive the data, and transmit it back to the host using **CDC_Transmit_FS** function.

STM32 USB CDC Setup (F103 Example)

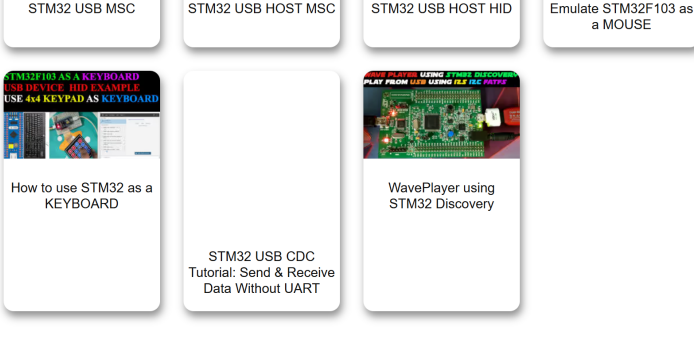
Below are some **definitions**, that will be used in the code

```
1 extern USB_HandTypeDef hUSBHostFS;
2 extern ApplicationTypeDef Appl1_state;
3 extern USB_StatusTypeDef USBResult;
4
5 #define RX_BUFF_SIZE 64 /* Max Received data 3KB */
6
7 uint8_t CDC_RX_Buffer[RX_BUFF_SIZE];
8 uint8_t CDC_TX_Buffer[RX_BUFF_SIZE];
9
10 typedef enum {
```

RESULT

The data is first transmitted by the **Host (STM32F411)**. It is then received by the **Device (STM32F103)** and transmitted back to **Host**.

The image below shows the data received by the **Host**.



As you can see above that the **CDC_RX_Buffer** have received the data (**DATA = 0**), that we transmitted to the device.

PROJECT DOWNLOAD

DOWNLOAD SECTION

Info

You can help with the development by DONATING Below.

To download the project, click the DOWNLOAD button.

Project FAQs

- Can **USB CDC** communication on **STM32** work simultaneously with **UART** or **SPI** without conflicts?
- How do I handle large data transfers over **USB CDC** without losing packets?
- Is it possible to switch dynamically between **USB Device** and **USB Host** modes at runtime?
- Can **USB CDC** be used for **real-time applications** like logging sensor data at high frequency?
- How can I make my **STM32 USB CDC** device appear as a custom serial device with its own driver name in **Windows**?

Search This Site

Search ...

Subscribe

Email

I accept the privacy policy

Subscribe

Categories

Select Category